

Velowen.art - Complete Project Documentation

Table of Contents

1. Project Overview
 2. Architecture Overview
 3. Modal Dashboard Explained
 4. Technical Stack
 5. Complete User Flow
 6. Key Components
 7. Why This Architecture?
 8. File Structure
 9. Environment Variables
 10. How 3D Gaussian Splatting Works
 11. Common Issues & Solutions
 12. Cost Breakdown
-

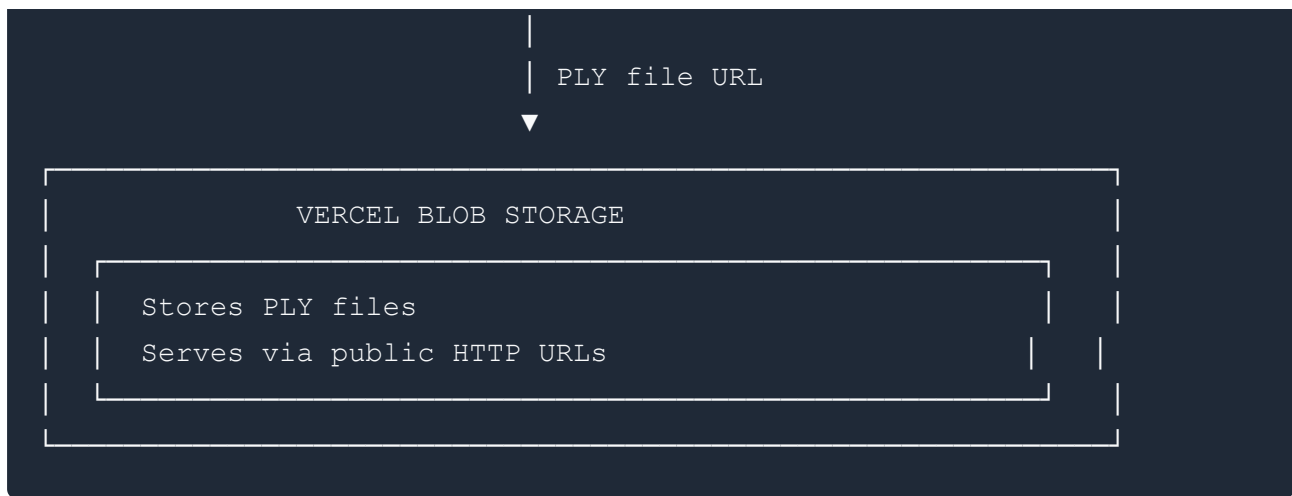
1. Project Overview

Velowen.art is a web application that converts a single 2D image into an interactive 3D scene using **3D Gaussian Splatting**. Users can upload an image or enter a text prompt, and the app generates a 3D scene that they can explore by flying through it.

Key Features: - Upload any image or enter a text prompt - AI converts image to 3D Gaussian Splats (PLY format) - Interactive 3D viewer with drag-to-rotate and scroll-to-fly controls - Minimalist, zen aesthetic homepage - Progressive reveal effect (white background → 3D scene)

2. Architecture Overview





Flow Summary: 1. User uploads image → Frontend converts to base64 2. Frontend calls `/api/generate-3d` → Vercel serverless function 3. Vercel calls Modal endpoint → GPU processes image 4. Modal returns PLY file → Vercel uploads to Blob Storage 5. Vercel returns public URL → Frontend loads in 3D viewer

3. Modal Dashboard Explained

What is Modal?

Modal is a serverless platform specifically designed for **GPU-accelerated Python applications**. Think of it like AWS Lambda, but for ML/AI workloads that need GPUs.

Key Features: - **No server management:** Just write Python code, Modal handles infrastructure - **GPU access:** Provides NVIDIA A100 GPUs on-demand - **Auto-scaling:** Containers start when needed, shut down when idle - **Pay-per-use:** Only pay for GPU time actually used - **Model caching:** Volumes persist model weights between invocations

What the Screenshot Shows

The screenshot shows the **Modal dashboard** for monitoring your deployed function.

Key Information: - **App Name:** `sharp-ml-app` - **Function:** `SharpModel.*` (the `generate` endpoint) - **Public URL:** `https://victorche0909--sharp-ml-app-sharpmodel-generate.modal.run` - **Status:** Shows function call history, timing, and logs

Metrics Explained: - **Containers: 0 live:** No active containers (scales to zero when idle) - **Calls: 0 running:** No active requests currently processing - **Function call results graph:** Shows activity over time (green bars = successful calls) - **Recent call:** Shows one successful call at `Dec 27, 01:35:36` - **Startup: 23.25s:** Cold start time (loading model into GPU memory) - **Execution: 12.05s:** Actual inference time (converting image to 3D) - **Status: 200:** Success

Why Modal? - You don't own a GPU → Modal provides A100 GPUs - You don't want to manage servers → Modal handles everything - You want to scale → Modal auto-scales containers - You want to pay only for usage → Modal charges per GPU-second

4. Technical Stack

Frontend (React + Vite)

Technologies: - **React:** UI framework for building components - **Vite:** Fast build tool and dev server - **Three.js:** 3D graphics library - **@mkellogg/gaussian-splats-3d:** Library for rendering Gaussian Splats - **Wouter:** Lightweight routing (like React Router) - **Framer Motion:** Animation library - **Lucide React:** Icon library

Why These? - React: Industry standard, great ecosystem - Vite: Much faster than Webpack - Three.js: Most popular 3D library - Gaussian Splats library: Specifically designed for this use case

Backend (Vercel Serverless)

Technologies: - **TypeScript/Node.js:** Type-safe JavaScript - **@vercel/node:** Runtime for serverless functions - **@vercel/blob:** File storage service

Why Vercel? - Zero-config deployment - Serverless functions (no server management) - Global CDN (fast worldwide) - Integrated with Git (auto-deploy on push)

AI/ML (Modal)

Technologies: - **Python 3.11:** Programming language - **PyTorch:** Deep learning framework - **Apple SHARP-ML:** Pre-trained model for image-to-3D conversion - **GPU:** NVIDIA A100 (40GB VRAM)

Why Modal? - GPU access without buying hardware - Handles all ML dependencies - Auto-scaling containers - Pay only for GPU time used

Deployment

Platforms: - **Vercel:** Hosts frontend + serverless functions - **Modal:** Hosts GPU inference - **Vercel Blob:** Stores PLY files

5. Complete User Flow (Step-by-Step)

Step 1: User visits velowen.art

```
Browser → Vercel → Serves React app (client/dist/index.html)
```

What happens: - Vercel serves the built React app - User sees homepage with search bar - 3D point cloud background renders - Season toggle available (winter/spring)

Step 2: User uploads image or enters prompt

```
User clicks search bar → Navigates to /image-to-3d  
User uploads image OR types prompt
```

What happens: - React Router (Wouter) navigates to `/image-to-3d` - User sees upload zone or prompt input - File validation checks size/type

Step 3: Frontend converts image to base64

```
// In image-to-3d.tsx  
const reader = new FileReader();  
reader.readAsDataURL(file);  
// Converts to: "data:image/jpeg;base64,/9j/4AAQSkZJRg..."  
const base64 = result.split(",")[1]; // Extract just the base64 part
```

Why base64? - JSON can't handle binary data - Base64 encodes binary as text - Can be sent in JSON body

Step 4: Frontend calls Vercel API

```
fetch("/api/generate-3d", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ image: base64 })  
})
```

What happens: - Browser sends HTTP POST request - Vercel routes to `api/generate-3d.ts` - Progress bar shows "Processing..."

Step 5: Vercel serverless function receives request

```
// api/generate-3d.ts  
const { image } = req.body; // base64 string
```

What happens: - Function starts (cold start ~100ms) - Extracts base64 image from request
- Validates image exists

Step 6: Vercel calls Modal endpoint

```
const modalEndpoint = 'https://victorche0909--sharp-ml-app-sharpmodel-gen  
const response = await fetch(modalEndpoint, {  
  method: 'POST',  
  body: JSON.stringify({ image })  
});
```

What happens: - Vercel makes HTTP request to Modal - Modal container starts (if idle) - Model loads into GPU memory (~23s cold start)

Step 7: Modal function processes image

```
# modal_inference.py
@modal.fastapi_endpoint(method="POST")
def generate(image: str):
    # 1. Decode base64 image
    image_bytes = base64.b64decode(image)

    # 2. Load image with PIL
    img = Image.open(io.BytesIO(image_bytes))

    # 3. Run SHARP-ML model (on GPU)
    gaussians = sharp_model.predict(img)

    # 4. Convert to PLY format
    ply_bytes = fast_save_ply_bytes(gaussians)

    # 5. Encode PLY as base64
    ply_base64 = base64.b64encode(ply_bytes).decode()

    # 6. Return JSON
    return {"success": True, "ply_base64": ply_base64}
```

What happens: - Decodes base64 → binary image - Loads image into memory - Runs neural network inference on GPU - Converts 3D Gaussians to PLY format - Encodes PLY as base64 for JSON response

Step 8: Vercel receives PLY data

```
const result = await response.json();
const plyBuffer = Buffer.from(result.ply_base64, 'base64');
```

What happens: - Vercel receives JSON response - Extracts `ply_base64` field - Converts base64 → binary buffer

Step 9: Vercel uploads to Blob Storage

```
const blob = await put(`outputs/${id}.ply`, plyBuffer, {
  access: 'public',
  contentType: 'application/x-ply'
});
const plyUrl = blob.url; // e.g., "https://xxx.public.blob.vercel-storage.com"
```

What happens: - Generates unique filename - Uploads PLY file to Vercel Blob - Gets public HTTP URL back

Why Blob Storage? - PLY files are large (10-50MB+) - Can't pass in URL (browser crashes)
- Blob Storage provides HTTP URLs

Step 10: Frontend receives URL

```
const result = await response.json();
setModelUrl(result.plyUrl);
setAppState("viewing");
```

What happens: - Frontend receives JSON with `plyUrl` - Updates state to show viewer - Progress bar shows 100%

Step 11: 3D viewer loads PLY

```
// GaussianViewer.tsx
const viewer = new GaussianSplats3D.Viewer({...});
await viewer.addSplatScene(modelUrl, {...});
```

What happens: - Creates Three.js renderer - Loads PLY file from URL - Parses Gaussian Splat data - Renders 3D scene

Step 12: User explores 3D scene

Controls: - **Drag:** Rotate camera - **Scroll:** Fly through scene - **Progressive reveal:** White background → 3D scene appears

6. Key Components

A. Frontend Components

`UIOverlay.tsx` (Homepage)

Purpose: Overlay on homepage with search bar

Features: - Search bar at top (like marble) - VELOWEN branding centered - Season toggle button (winter/spring) - Navigates to `/image-to-3d` on search focus

`image-to-3d.tsx` (Main Page)

Purpose: Main page for image upload and 3D viewing

State Management: - `appState` : "upload" | "processing" | "viewing" | "error" - `processingStage` : "uploading" | "processing" | "generating" | "complete" - `modelUrl` : URL of PLY file - `previewUrl` : Preview of uploaded image

Features: - Tab switcher (Upload / Prompt) - Progress tracking - Error handling - 3D viewer integration

`GaussianViewer.tsx` (3D Viewer)

Purpose: Renders 3D Gaussian Splats

Technologies: - `@mkkellogg/gaussian-splats-3d` : Main library - Three.js: 3D rendering - OrbitControls: Camera controls

Features: - White background (progressive reveal) - Drag to rotate - Scroll to fly through - Loading progress indicator

`ImageUpload.tsx`

Purpose: File upload component

Features: - Drag-and-drop zone - File validation (max 4.5MB, PNG/JPG/WEBP) - Preview with pixelation effect - Error messages

`PromptInput.tsx`

Purpose: Text prompt input

Features: - Textarea for prompts - Example chips (quick prompts) - Submit on Enter key - Auto-focus

`ProcessingStatus.tsx`

Purpose: Shows generation progress

Features: - Progress bar - Stage indicators (uploading → processing → generating) - Time estimates - Error display

B. Backend Components

`api/generate-3d.ts` (Vercel Serverless)

Purpose: Main API endpoint

Flow: 1. Receives base64 image 2. Calls Modal endpoint 3. Gets PLY file back 4. Uploads to Vercel Blob Storage 5. Returns public URL

Error Handling: - Validates image exists - Handles Modal errors - Validates Blob Storage token

`modal_inference.py` (Modal Function)

Purpose: GPU inference endpoint

Setup: - Container image with all dependencies - Model loading on startup - GPU-optimized postprocessing

Endpoints: - `POST /` : Receives image, returns PLY

Performance: - Cold start: ~23s (model loading) - Warm inference: ~12s (actual processing)

C. Configuration Files

`vercel.json`

Purpose: Vercel deployment config

Config: - Build command: `npm run build` - Output directory: `dist/public` -
Framework: `vite` - Rewrites: All routes → `index.html` (SPA)

`package.json`

Purpose: Dependencies and scripts

Key Dependencies: - React, Vite, Three.js - Gaussian Splats library - Vercel packages

Scripts: - `dev` : Start dev server - `build` : Build for production - `preview` : Preview production build

7. Why This Architecture?

Why Vercel?

Benefits: - **Zero-config deployment:** Just push to Git - **Serverless functions:** No server management - **Global CDN:** Fast worldwide - **Integrated:** Works with Git, GitHub, etc.

Trade-offs: - Function timeout limits (10s hobby, 60s pro) - Cold starts (~100ms)

Why Modal?

Benefits: - **GPU access:** A100 GPUs without buying hardware - **Auto-scaling:** Containers start/stop automatically - **Pay-per-use:** Only pay for GPU time - **Handles ML:** All dependencies managed

Trade-offs: - Cold starts (~23s for model loading) - Cost per GPU-second

Why Vercel Blob Storage?

Benefits: - **Large files:** Handles 10-50MB+ PLY files - **Public URLs:** HTTP URLs for viewer - **No memory issues:** Avoids browser crashes

Trade-offs: - Additional cost (~\$0.15/GB/month) - Requires token setup

Why React + Vite?

Benefits: - **Fast development:** Hot module replacement - **Modern tooling:** ES modules, TypeScript - **Great performance:** Optimized builds

Trade-offs: - Learning curve (if new to React) - Bundle size (mitigated by code splitting)

8. File Structure

9. Environment Variables

Vercel Dashboard

Required: - `MODAL_ENDPOINT_URL` : Your Modal function URL - Example:

`https://victorche0909--sharp-ml-app-sharpmodel-generate.modal.run` -

`BLOB_READ_WRITE_TOKEN` : Vercel Blob access token - Get from: Vercel Dashboard → Storage → Blob → Create Store

How to Set: 1. Go to Vercel Dashboard 2. Select your project 3. Settings → Environment Variables 4. Add each variable 5. Redeploy

Modal

Volumes: - `sharp-model-cache` : Persists model weights between invocations - Created automatically on first deploy

10. How 3D Gaussian Splatting Works

What is Gaussian Splatting?

Traditional 3D: - Meshes (triangles) - Point clouds (dots) - Voxels (3D pixels)

Gaussian Splatting: - Thousands of 3D Gaussians (ellipsoids) - Each Gaussian has: -

Position (x, y, z) - **Rotation** (quaternion) - **Scale** (3D size) - **Color** (RGB) - **Opacity** (alpha)

Why Gaussian Splatting? - **Photorealistic:** Captures fine details - **Efficient:** Renders fast on GPU - **View-dependent:** Looks different from different angles - **Progressive:** Can load gradually

How SHARP-ML Creates It

Process: 1. **Input:** Single 2D image 2. **Neural Network:** Predicts 3D structure - Depth estimation - Surface normals - Geometry reconstruction 3. **Gaussian Generation:** Creates thousands of Gaussians - Positioned in 3D space - Colored from input image - Optimized for view synthesis 4. **Export:** Saves as PLY file

Model Details: - Pre-trained by Apple - Trained on diverse images - Optimized for single-image input

How the Viewer Renders It

Process: 1. **Load PLY:** Parses file format 2. **Parse Gaussians:** Extracts position, rotation, scale, color, opacity 3. **GPU Rendering:** Uses shaders to render splats - Each Gaussian rendered as 2D splat - Alpha blending for transparency - Depth sorting for correct layering 4. **Progressive Loading:** Loads gradually (white → scene)

Library: - `@mkkellogg/gaussian-splats-3d` - Built on Three.js - Optimized WebGL shaders

11. Common Issues & Solutions

Issue: Browser crashes when loading PLY

Symptoms: - Browser freezes - "Out of Memory" error - Tab crashes

Cause: - PLY file passed as base64 in URL - Browser runs out of memory

Solution: - Use Vercel Blob Storage - Serve PLY via HTTP URL - Viewer loads from URL, not data

Issue: Slow generation (35+ seconds)

Symptoms: - Progress bar stuck - Long wait times

Cause: - Cold start on Modal (~23s) - Model loading into GPU memory

Solution: - Normal for first call - Subsequent calls faster (~12s) - Consider keeping container warm (pro plan)

Issue: Viewer shows black screen

Symptoms: - Viewer loads but shows nothing - Progress bar completes but no scene

Cause: - PLY URL format incorrect - Viewer library doesn't support data URLs - CORS issues

Solution: - Use HTTP URLs (Vercel Blob) - Check browser console for errors - Verify PLY file is valid

Issue: "Vercel Blob storage not configured"

Symptoms: - Error message on generation - Function fails

Cause: - `BLOB_READ_WRITE_TOKEN` not set

Solution: 1. Go to Vercel Dashboard 2. Storage → Blob → Create Store 3. Copy token 4. Settings → Environment Variables → Add token 5. Redeploy

Issue: Modal function not found

Symptoms: - "404 Not Found" error - Function doesn't exist

Cause: - Function not deployed - Wrong URL

Solution: 1. Deploy Modal function: `modal deploy backend/modal_inference.py` 2. Copy URL from Modal dashboard 3. Set `MODAL_ENDPOINT_URL` in Vercel

12. Cost Breakdown

Vercel

Free Tier: - 100GB bandwidth/month - Unlimited serverless function invocations - 100GB-hours compute time

Blob Storage: - ~\$0.15/GB/month - Typical PLY file: 10-50MB - 1000 generations ≈ 10-50GB ≈ \$1.50-7.50/month

Total Vercel Cost: - **Free tier:** \$0 (for most use cases) - **Blob Storage:** ~\$1-10/month (depending on usage)

Modal

Pricing: - A100 GPU: ~\$1.50/hour - Billed per GPU-second - Typical generation: ~35 seconds

Cost Calculation: - 35 seconds = 0.0097 hours - $0.0097 \times \$1.50 = \sim\0.015 per generation

Monthly Estimate: - 100 generations = \$1.50 - 1000 generations = \$15 - 10,000 generations = \$150

Total Cost Estimate

Low Usage (100 generations/month): - Vercel: \$0 (free tier) - Blob Storage: ~\$1 - Modal: ~\$1.50 - **Total: ~\$2.50/month**

Medium Usage (1000 generations/month): - Vercel: \$0 (free tier) - Blob Storage: ~\$5 - Modal: ~\$15 - **Total: ~\$20/month**

High Usage (10,000 generations/month): - Vercel: \$0-20 (may need pro) - Blob Storage: ~\$50 - Modal: ~\$150 - **Total: ~\$200-220/month**

Summary

Velowen.art is a complete full-stack application that:

1. **Takes a 2D image** → User uploads or prompts
2. **Sends to Modal** → GPU converts to 3D
3. **Stores result** → Vercel Blob Storage
4. **Displays in viewer** → Interactive 3D scene

Architecture: - **Frontend:** React + Vite (fast, modern) - **Backend:** Vercel Serverless (zero-config) - **AI:** Modal (GPU inference) - **Storage:** Vercel Blob (large files)

Key Technologies: - 3D Gaussian Splatting (photorealistic 3D) - Apple SHARP-ML (image-to-3D model) - Three.js (3D rendering) - Serverless architecture (scalable, cost-effective)

The Modal dashboard shows: - Function activity (calls, timing, logs) - Container status (live/idle) - Performance metrics (startup, execution time)

This architecture allows you to run GPU-accelerated AI without owning hardware, scale automatically, and pay only for what you use.

Document Version: 1.0

Last Updated: December 2024

Project: Velowen.art

Author: Project Documentation